Contents

Guide 2 — The Recommendation Engine (simple, and how to defend it) **1**
1. The idea, in three sentences 1
2. The approach, and why 1
3. How it works — the steps 1
4. The formula, on a real example 1
5. The files (one line each) 2
6. Jury questions — ready answers 3

Guide 2 — The Recommendation Engine (simple, and how to defend it)

A short, plain guide to how Mobile Match Algeria recommends plans: the idea, the approach and why, how it works, the formula on a real example, the files, and ready answers for the jury.

---

1. The idea, in three sentences

The user tells us their **budget**, how much **data** they need, optionally their preferred operator / type / network, and **what matters most** to them (price or data). The engine throws away every plan that doesn't fit the hard limits, scores the rest on how well they match, and returns the **top 3**, ranked. It uses no AI and no other users' data, only the one profile the user gives us.

---

2. The approach, and why

We use **content-based filtering with a tiered ranking** — not collaborative filtering and not AI. Why:

- A brand-new app has **no history** of what users clicked or bought, so an AI or a "people like you also chose…" model has nothing to learn from. This is the classic **cold-start problem**.
- Our catalogue is **small (~100 plans)** and every plan is fully described by numbers (price, data, minutes, SMS). Matching those numbers against the user's stated needs is enough, and it is instant.

This is a known, cited technique (lexicographic preference ordering, Fishburn 1974), not something we invented.

In one line: *filter out what doesn't fit, score what's left against the user's own needs, then rank by what they said matters most.*

---

3. How it works — the steps

1. **Collect the profile** — budget, data need, optional operator / type / network, and 1–2 priorities ranked (price, data).
2. **Hard filters** — drop any plan over budget, or with the wrong operator / type / network. (Budget is a ceiling: a plan you can't afford is never shown.)
3. **Score two merits** (each from 0 to 1) on every surviving plan:
   - **price merit** — how cheap it is versus the budget (cheaper → closer to 1)
   - **data merit** — how well its data covers your need (covers it → 1)

4. **Tier and rank** — group plans into 5 bands by your #1 priority, then order inside each band by your #2 priority.
5. **Return the top 3**, each with the **savings** it leaves versus your budget.

---

4. The formula, on a real example

This is the part worth memorising. Two merits, each between 0 and 1:

```
price merit = 1 − (price / budget)     (clamped to 0..1; cheaper is better)
data merit  = data / need              (clamped to 0..1; unlimited = 1)
```

Then, when the user ranks two priorities (say #1 = price, #2 = data):

```
tier  = round(price merit × 4) / 4     ->  one of {0, 0.25, 0.5, 0.75, 1}   (5 bands)
score = tier + 0.24 × data merit
rank by score, highest first
```

What a "tier" is. A merit is a decimal from 0 to 1. We **round it to the nearest 0.25**, onto one of five rungs (0, 0.25, 0.5, 0.75, 1) — that is all `round(m × 4)/4` does. A *tier* is a rung, and plans on the same rung count as **equally good** on that criterion, so the 2nd priority can decide between them. (Why round at all? Otherwise a plan 1 DA cheaper would always beat a slightly pricier one even with far less data; rounding groups "close enough" plans so the 2nd priority has room to act.)

Worked example. Budget = 2000 DA, need = 10 GB, priority #1 = price, #2 = data.

Plan	Price	Data	Price merit	Tier	Data merit	Score
A	1000 DA	8 GB	0.50	0.50	0.80	0.50 + 0.24×0.80 = **0.692**
B	1100 DA	15 GB	0.45	0.50	1.00	0.50 + 0.24×1.00 = **0.740**
C	600 DA	4 GB	0.70	0.75	0.40	0.75 + 0.24×0.40 = **0.846**

Ranking: C → B → A.

What to say about it: - **C wins** because it sits in a higher **price tier** (0.75). The user said price matters most, so a cheaper plan beats a pricier one **across bands** — no matter how much data the pricier plan has. - **A and B are in the same price tier** (both 0.50), so there the **data merit** breaks the tie, and B (15 GB) beats A (8 GB). - This is the whole point: the **#2 priority only reorders within a band; it can never overtake the #1 priority** — because 0.24 is smaller than one tier step (0.25).

If the user picks only **one** priority, we skip the tiers and rank directly by that merit.

---

5. The files (one line each)

- **src/lib/recommendation.ts** — the engine: the hard filters, the two merits, the tier formula, and the top-N. The whole formula above lives here.

- **src/app/api/recommendations/route.ts** — the API the page calls; it validates the incoming profile and is rate-limited (60 requests/min).
- **src/app/recommend/page.tsx** — the screen: the profile form and the ranked result cards (each showing its savings).
- **src/app/profile/page.tsx** — where a logged-in user saves their profile, so the daily job can recommend for them automatically.

Note: each plan also gets a 0–100 "fitness score" (budget, data, voice, value…), but we use it **only as a tiebreaker** and we **don't show a match %** on the cards — we show the **rank** (1st / 2nd / 3rd) and the **savings**, which users find clearer.

---

6. Jury questions — ready answers

Q — Why not AI or machine learning? AI recommenders learn from large amounts of past behaviour (what thousands of users clicked or bought). A new app has none of that — the cold-start problem. With nothing to learn from, a model would do no better than guessing, so we use the data we actually have: the user's own stated profile.

Q — Why content-based and not collaborative filtering, like Netflix? Collaborative filtering means "people like you also chose X" — it needs a history of many users' choices, which we don't have at launch. Content-based filtering matches the plan's own attributes (price, data…) to the user's needs, which works from day one. The literature confirms content-based is the standard answer to cold-start.

Q — Why tiers instead of one weighted score (e.g. 0.6·price + 0.4·data)? We tried a weighted sum first and it failed: it lets a great data score **hide** a bad price score, so a plan the user can barely afford could outrank a cheaper one — which contradicts "price matters most to me". Tiers fix that: the #1 priority decides the band, and the #2 can only reorder inside a band. We rejected five designs before this one (documented in Chapter 3, §3.4).

Q — Did you invent this formula, or is it researched? The **method** is researched and cited — lexicographic preference ordering with bounded relaxation, from operations research (Fishburn 1974, Roberts 1979; modern treatment by Goswami, Mitra & Sen 2021). The **specific numbers** (5 tiers, the 0.24 weight) are our own engineering choices, each justified by the maths below — the thesis says clearly that the contribution is *the adaptation*, not the algorithm.

Q — Isn't the formula arbitrary? Why 0.24 and 0.25? 0.25 gives 5 tiers (0, 0.25, 0.5, 0.75, 1) — enough bands to separate cheap / medium / expensive without splitting hairs on ~100 plans (we tried a finer step of 0.1 and it broke: each band held ~1 plan, so the 2nd priority never acted). 0.24 is deliberately just **under** 0.25: the 2nd priority adds at most `0.24 × 1 = 0.24`, which is less than one tier step (0.25), so its biggest possible push still can't reach the next tier — that is what guarantees the #1 priority always wins across bands. **Proof:** a tier-0.50 plan with perfect data scores 0.50 + 0.24 = 0.74; a tier-0.75 plan with zero data scores 0.75, and 0.75 > 0.74, so the higher tier still wins. The 0.01 gap absorbs floating-point rounding.

Q — What if no plan fits the budget or filters? Budget is a hard ceiling, so if nothing fits, the list is simply empty and the screen says so. We never show a plan the user can't afford; they can raise the budget or relax a filter.

Q — Why only the top 3? The goal is a decision, not a catalogue. Three ranked choices is enough to compare without overwhelming; the full list is always on the Offers page.

Q — Can a user game or trick it? There is nothing to game — the result is a deterministic function of the user's own profile and the live catalogue. No clicks, no popularity, no ads influence it.

Q — Is this a real algorithm or just sorting? It is an applied form of **lexicographic preference ordering with bounded relaxation**, a multi-criteria decision method from operations research (Fishburn 1974, Roberts 1979). The “tier, then refine” rule **is** that method; sorting is only the final step.

Q — How is “savings” computed? Simply budget − plan price (never negative). It shows how much of the monthly budget the plan leaves unspent, which is also why a cheaper plan in the same tier is preferred.

Q — Why is budget a hard limit but data only a score? Money is a real constraint — a plan over budget is useless, so it is filtered out entirely. Data is a preference — a plan slightly under your stated need may still be worth showing, so it is scored, not filtered.

Next guide: the search engine (the token parser + FTS5). Tell me when you want it.